

Exp 3: ADJUSTING LED BRIGHTNESS BY USING POTENTIOMETER

Name: Pradip Maity
CDSID: PMAITY



Table of Contents

S.No.		Page No
1	Introduction	3
2	Scope	3
3	Prerequisite	3
4	Connection Diagram	3
5	Working Principle	4
6	Application Code	4
7	Output	5
8	Hardware Image	5
9	Appendix	6

1. Introduction:

This project demonstrates how to control the brightness of an LED using a potentiometer with the Raspberry Pi Pico. A potentiometer is a variable resistor that provides an adjustable voltage output. An ADC (Analog-to-Digital Converter) is used to convert this analog voltage into a digital value. PWM (Pulse Width Modulation) is a technique used to control the intensity of devices like LEDs by varying the duty cycle of the signal. The analog voltage from the potentiometer is read using ADC and then used to adjust the PWM signal, which controls the LED brightness.

2. Scope:

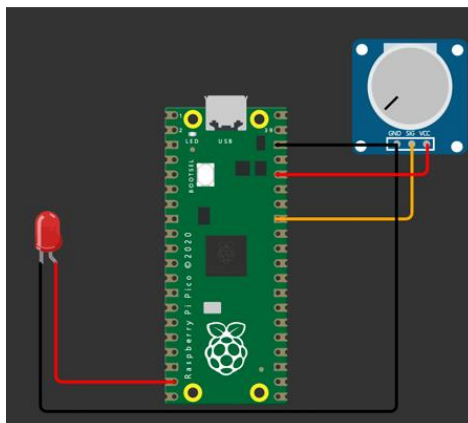
This project provides an understanding of analog input processing and PWM output control in a microcontroller. It demonstrates how real-world analog signals can be used to control digital outputs. These concepts are widely used in embedded systems, robotics, and automation for sensor-based control applications.

3. Prerequisite:

Hardware: Raspberry Pi Pico Microcontroller, USB Connector, Breadboard, LED, Potentiometer, Jumper Wires

Software: VS Code (with Pico SDK), PuTTY Serial Console (in Windows)

4. Connection Diagram:



5. Working Principle:

The potentiometer is connected to an ADC pin of the Raspberry Pi Pico, which reads the varying analog voltage as the knob is adjusted. The ADC converts this analog voltage into a digital value.

This digital value is then scaled and applied to a PWM output pin connected to the LED. PWM works by varying the duty cycle of the signal, which controls the amount of power delivered to the LED.

As the potentiometer value changes, the ADC reading changes accordingly. This value is mapped to a corresponding PWM value, which adjusts the LED brightness. Higher ADC values result in higher PWM duty cycles, making the LED brighter, while lower values reduce brightness.

The process runs continuously, allowing real-time adjustment of LED brightness based on the potentiometer position.

6. Application code:

```
#include <stdio.h>
#include "pico/stdlib.h"
#include "hardware/adc.h"
#include "hardware/pwm.h"

#define ADC_PIN 26
#define LED_PIN 15

int main() {
    stdio_init_all();
    sleep_ms(2000);
    adc_init();
    adc_gpio_init(ADC_PIN);
    adc_select_input(0);
    gpio_set_function(LED_PIN, GPIO_FUNC_PWM);
    uint slice_num = pwm_gpio_to_slice_num(LED_PIN);
    pwm_set_wrap(slice_num, 65535);
    pwm_set_enabled(slice_num, true);
    printf("ADC to PWM LED Control Started\n");

    while (1) {
        uint16_t adc_value = adc_read();
        uint16_t pwm_value = adc_value * 16;
        pwm_set_gpio_level(LED_PIN, pwm_value);
        printf("ADC: %d  PWM: %d\n", adc_value, pwm_value);
        sleep_ms(100);
    }
}
```

7. Output:

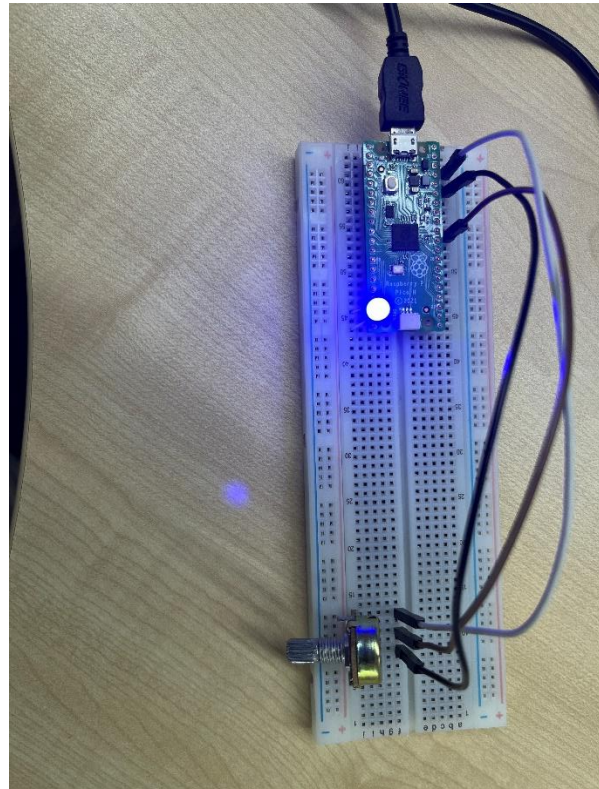
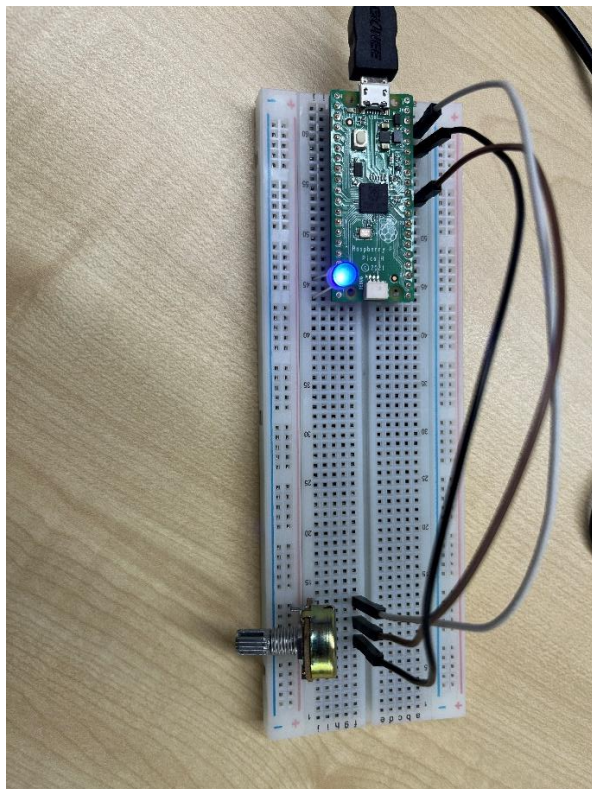
PuTTY (inactive)

ADC: 218	PWM: 3488
ADC: 218	PWM: 3488
ADC: 216	PWM: 3456
ADC: 218	PWM: 3488
ADC: 217	PWM: 3472
ADC: 221	PWM: 3536
ADC: 233	PWM: 3728
ADC: 331	PWM: 5296
ADC: 450	PWM: 7200
ADC: 562	PWM: 8992
ADC: 665	PWM: 10640
ADC: 763	PWM: 12208
ADC: 833	PWM: 13328
ADC: 908	PWM: 14528
ADC: 961	PWM: 15376
ADC: 1017	PWM: 16272
ADC: 1089	PWM: 17424
ADC: 1152	PWM: 18432
ADC: 1204	PWM: 19264
ADC: 1260	PWM: 20160
ADC: 1298	PWM: 20768
ADC: 1370	PWM: 21920
ADC: 1424	PWM: 22784
ADC: 1455	PWM: 23280

PuTTY (inactive)

ADC: 1424	PWM: 22784
ADC: 1455	PWM: 23280
ADC: 1517	PWM: 24272
ADC: 1554	PWM: 24864
ADC: 1621	PWM: 25936
ADC: 1684	PWM: 26944
ADC: 1740	PWM: 27840
ADC: 1812	PWM: 28992
ADC: 1869	PWM: 29904
ADC: 1927	PWM: 30832
ADC: 1982	PWM: 31712
ADC: 2054	PWM: 32864
ADC: 2122	PWM: 33952
ADC: 2195	PWM: 35120
ADC: 2271	PWM: 36336
ADC: 2331	PWM: 37296
ADC: 2385	PWM: 38160
ADC: 2479	PWM: 39664
ADC: 2552	PWM: 40832
ADC: 2634	PWM: 42144
ADC: 2740	PWM: 43840
ADC: 2829	PWM: 45264
ADC: 2957	PWM: 47312
ADC: 3051	PWM: 48816

8. Hardware Image:



9. Appendix:

References

- Official Documentation for Raspberry Pi Pico and Pico W
- Technical Datasheet for Raspberry Pi Pico
- GitHub Repository for Raspberry Pi Pico SDK
- PuTTY Serial Console Documentation